

Globally Optimized Matchmaking in Online Games

Qilin Deng*
NetEase Fuxi AI Lab
Hangzhou, Zhejiang, China
qlqldeng@163.com

Hao Li*
NetEase Fuxi AI Lab
Hangzhou, Zhejiang, China
lihao01@corp.netease.com

Kai Wang
NetEase Fuxi AI Lab
Hangzhou, Zhejiang, China
wangkai02@corp.netease.com

Zhipeng Hu
NetEase Fuxi AI Lab
Hangzhou, Zhejiang, China
zphu@corp.netease.com

Runze Wu†
NetEase Fuxi AI Lab
Hangzhou, Zhejiang, China
wurunze1@corp.netease.com

Linxia Gong
NetEase Fuxi AI Lab
Hangzhou, Zhejiang, China
gonglinxia@corp.netease.com

Jianrong Tao
NetEase Fuxi AI Lab
Hangzhou, Zhejiang, China
hztaojianrong@corp.netease.com

Changjie Fan
NetEase Fuxi AI Lab
Hangzhou, Zhejiang, China
fanchangjie@corp.netease.com

Peng Cui
Tsinghua University
Beijing, China
cui@tsinghua.edu.cn

ABSTRACT

As one of the core components of online games, matchmaking is the process of arranging multiple players into matches, where the quality of matchmaking systems directly determines player satisfaction and further affects the life cycle of game products. With the number of candidate players increases, the number of possible match combinations grows exponentially, which makes the current implementation for multiplayer matchmaking can only obtain locally optimal arrangement in an inefficient fashion. In this paper, we focus on the globally optimized matchmaking problem, in which the objective is to decide an optimal matching sequence for the queuing players. To tackle this challenging problem, we propose a novel data-driven matchmaking framework, called *GloMatch*, based on machine learning principles. Through transforming the matchmaking problem into a sequential decision problem, we solve it with the help of an effective policy-based deep reinforcement learning algorithm. Quantitative experiments on simulation and online game environments demonstrate the effectiveness of the presented framework.

CCS CONCEPTS

• **Computing methodologies** → **Markov decision processes; Reinforcement learning**; • **Information systems** → **Massively multiplayer online games; Data mining**.

*Both authors contributed equally to this research.

†Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

KDD '21, August 14–18, 2021, Virtual Event, Singapore

© 2021 Association for Computing Machinery.

ACM ISBN 978-1-4503-8332-5/21/08...\$15.00

<https://doi.org/10.1145/3447548.3467074>

KEYWORDS

matchmaking; online games; deep learning; markov decision process; reinforcement learning

ACM Reference Format:

Qilin Deng, Hao Li, Kai Wang, Zhipeng Hu, Runze Wu, Linxia Gong, Jianrong Tao, Changjie Fan, and Peng Cui. 2021. Globally Optimized Matchmaking in Online Games. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '21), August 14–18, 2021, Virtual Event, Singapore*. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3447548.3467074>

1 INTRODUCTION

In recent years, online games have attracted millions of players. For example, popular Multiplayer Online Battle Arena (MOBA) games such as *League of Legends* (LOL) or *Defense of the Ancients 2* (DOTA2) have become one of the most played genres, and they also become important platforms for esports tournaments, distributing millions of dollars in prizes. Among these games, one of the fundamental gameplay designs is player-versus-player (PvP), a game mode in which multiple players directly engage in competition. For example, in DOTA2, two teams of five players compete against each other to destroy the enemy's base.

As one of the core modules of these online games, matchmaking is the process of arranging multiple players into PvP matches. In competitive games and sports, reasonable and fair matchmaking mechanisms can bring players a more enjoyable and exciting game experience. In the context of online games, the process of typical matchmaking service involves: first, online players request the gaming service and indicate their availability; then, the matchmaking system (matchmaker) receives matching requests and forms teams for queuing players; finally, the system brings up every two teams into one combat match. During this process, the typical matchmaking method will rank all the candidate players in the player pool by specific criteria. Then, according to this rank order, it will select players to form two teams for the first match, and then continue to select players for the second match, etc. This greedy strategy is reasonable or even optimal in some situations, e.g., the Elo rating system [2, 9], where a player's skill rating is represented by a single

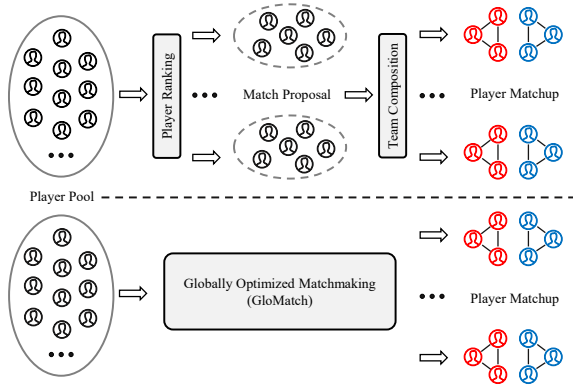


Figure 1: Comparison of the traditional matchmaking system (top part) with our proposed matchmaking system GloMatch (bottom part).

number. The matchmaking system can rank the players according to the Elo ratings, which is actually the optimal result, assuming that it is accurate to use a number to represent the player’s overall skill. The Elo ratings is now widely adopted in 1-vs-1 zero-sum games such as *chess* and *go*.

However, with the games being designed more sophisticated, a single scalar rating is oversimplified to reflect the player’s diverse abilities in the competition [4, 5, 8, 12, 28], e.g., the solo ability, team battle ability, line pushing awareness, gank ability¹, etc. for players in MOBA games. Moreover, compared to typical 1-vs-1 matches, the matchmaking problem is more complex in team-oriented game scenarios, where team composition is a central factor in determining the effectiveness of competing teams. In such cases, the combat effectiveness of a team as a whole cannot be simply represented by a linear combination of individual abilities. Therefore, a straightforward extension on Elo ratings by adding up skills for all team members [13] cannot work well due to its inability to capture the team synergy effect among players.

To alleviate this problem, there is a growing research interest in applying machine learning methods to the matchmaking system [2, 4–6, 8, 12, 13, 23]. These methods basically follow a two-stage paradigm: a player ranking stage followed by a team composition stage. The player ranking stage is responsible for setting a selection priority for all candidate players, which is usually done by hand-crafted rules. In the team composition stage, they first select the players involved in a match according to this priority, then choose the best way to divide the players into two competing teams by predicting the balance of each division. Although these methods are meaningful progress, the application of machine learning is only in the team composition stage. Such solutions are at best locally optimal in the sense of a single-match matchmaking, rather than globally optimal in the sense of the entire player pool.

In this paper, we focus on the globally optimized matchmaking problem, i.e., how to optimally divide a pool of (hundreds) players into multiple matches. Figure 1 is a comparative demonstration of the traditional two-stage method and the globally optimized

method. Although it can be formalized as a combinatorial optimization problem on the set of candidate players, there are two difficulties in this problem: firstly, as the number of candidate players involved in a match increases, the number of possible team compositions grows exponentially. More importantly, greedy strategies or hand-crafted rules, which may work in the Elo-ratings-like matchmaking scenarios, fail to globally optimize the matching quality of the matchmaking system in the sense of the player pool.

In order to tackle these difficulties, we propose a machine learning matchmaking method, called **GloMatch** for **g**lobally **o**ptimized **m**atchmaking. Inspired by recent progress on the cross-over study of combinatorial optimization and deep learning, especially the integration with deep reinforcement learning, we introduce an end-to-end globally optimized matchmaking framework. To this end, we transform the matchmaking problem into a sequential decision problem, and formalize the matchmaking process as a Markov Decision Process (MDP), in which the state space and reward function are specially designed for our matchmaking problem. Based on the MDP formulation, we further solve the problem by an effective policy-based deep reinforcement learning algorithm. Our method can efficiently search in the exponential player combination space to optimize the gross player satisfaction. Overall, the main contributions of this paper are summarized as follows:

- We explore the ubiquitous yet challenging problem of global matchmaking optimization for a pool of players in the context of online games.
- We propose a novel reinforcement learning framework for multi-player matching, through which globally optimized matchmaking can be realized in a scalable and efficient way.
- Extensive simulation experiments on the industrial game datasets are conducted to verify the effectiveness of the proposed method. Furthermore, we deployed the matchmaking system online and verified the improvement of matchmaking quality through online experiments.

2 RELATED WORK

Many existing matchmaking algorithms originated from pairwise comparison [25]. The seminal work of the Bradley-Terry model [3], a probability model that can predict the outcome of a paired comparison, naturally extends to matchup prediction. In the Bradley-Terry model, each player’s strength is represented by a single real number. The famous Elo rating system [9], which was adopted by the World Chess Federation (FIDE) [10], is a statistical rating system originally developed for rating chess players. Generally, it is a method for calculating the relative skill levels of players in zero-sum games such as chess. The key idea behind the Elo system is to model the probability of the possible game outcomes as a function of the two players’ skill ratings. It is worth noting that the Elo ratings is a comparative rating, and is valid only within the rating pool where it was established. As an improvement of the Elo ratings, the Glicko system [11], a Bayesian ranking rating system, introduces the idea of modeling the belief about a player’s skill as a Gaussian distribution characterized by a mean and a variance. Glicko’s principal contribution is “rating reliability” (called RD, for rating deviation), which itself decreases after playing a game, but it will increase slowly over time of inactivity. However, in the multiplayer online

¹A common tactic in the game, which refers to two or more heroes fighting side by side to attack, outflank, and siege enemy heroes.

games, game outcomes often refer to teams of players. All of the Bradley-Terry model, the Elo system, and the Glicko system are not initially applicable to team-oriented games. To address such challenges, several efforts have been made to extend the basic rating system [13–15, 17]. For example, the TrueSkill² [13] and its extensions [7, 18] generalize the Elo system to games with flexible numbers of players and teams, in a principled Bayesian framework. Such methods differ significantly from ours: they are probabilistic methods that model a player’s skill as a scalar random variable and perform inference based only on match results (in particular they ignore player attributes).

There are several works [5, 6, 8, 12] take player profiles (features) into account, in addition to the historical match results. However, both of the methods presented in [5] and [6] are only suitable for pairwise comparison instead of team comparison. The models proposed in [8, 12] complete the global matchmaking process by combining random sampling or heuristic sorting rules and outcome prediction models, which can be regarded as locally optimized matchmaking with respect to players involved in one match, rather than global optimization with respect to the matching pool. Different from the aforementioned works, the framework described in this paper is a general data-driven neural optimization framework. It considers the modeling of global optimal matchmaking in team-oriented games and generalizes skill-based player ranking methods for the two-player game with more expressive power. It can also utilize game outcomes of historical matches and player profiles, no matter whether the profiles are discrete or continuous.

Despite the efforts made into modeling matchmaking problem from the perspective of machine learning, the solution is still largely under-explored from the perspective of global optimization. In this paper, we treat the global matchmaking problem as a combinatorial optimization problem, where the objective is to generate (choose) a matching sequence to maximize a utility objective function. Further, inspired by recent progress on the neural combinatorial optimization method [1, 20, 27], we transform the combinatorial optimization problem into a sequential decision problem, and resort to modern RL algorithms to solve this challenging problem within an end-to-end neural optimization framework.

3 PROBLEM FORMULATION

Let $\mathcal{P} = \{p_i | i = 1, 2, \dots, N\}$ be the player pool that one could select from and $N = |\mathcal{P}|$ be the number of all candidate players. We also assume that each player is associated with a profile vector x , which can describe various aspects of the player’s ability, i.e., we have $\mathcal{F} = \{x_i \in \mathbb{R}^d | p_i \in \mathcal{P}\}$. A team $T = \{p_i \in \mathcal{P} | i = 1, 2, \dots, K\}$ is any (unordered) player combination selected from the player pool $\mathcal{P}$, and K denotes the team size (number of players in a team). Supposing that N players can form M matches (i.e., $N = 2KM$), our goal is to generate an optimal matching sequence $\mathcal{M} = \{T_a^m, T_b^m\}_{m=1}^M$, with the greatest gross player utilities measured by match level³:

$$\mathcal{U}(\mathcal{M}) = \sum_m \mathcal{U}_m(T_a, T_b) \quad (1)$$

²<https://trueskill.org/>

³Here we focus on situations where only two teams with equal size face each other, i.e., $|T_a| = |T_b| = K$, however, our framework could be generalized to more generic settings as well.

where $\mathcal{U}_m(T_a, T_b)$ denotes the utility function when team T_a and T_b compete with each other in a match $m \in \{1, 2, \dots, M\}$. Here, the definition of utilities is general and could be implemented based on specific matchmaking goals, e.g., the balance of abilities between the two teams in a match, or more implicit indicators such as whether players continue to play after completing a match⁴, or a combination of certain indicators that reflects the trade-offs of various goals the matchmaking system wants to achieve.

Obviously, this problem could be modeled as a combinatorial optimization problem. Corresponding to our matchmaking case, we need to search the entire combination space of all players in $\mathcal{P}$ to find a player permutation such that the optimal total value $\mathcal{U}(\mathcal{M})$ could be achieved. However, the optimization problem described here involves a special utilities objective $\mathcal{U}_m(T_a, T_b)$, which is usually a highly non-linear function, considering the complex interaction effect between the competing teams and players within a team in the match. This non-linearity makes our problem unable to be formulated as linear programming and further solved by modern operations research tools.

4 PRELIMINARY STUDY

Given the formal definition of the global matchmaking problem above, in this section, we propose a data-driven neural optimization framework to solve this challenging problem. In general, we transform it into a sequential decision problem, and develop a framework with the capability of solving the combinatorial optimization problems using a Deep Reinforcement Learning (DRL) algorithm.

4.1 Outcome Prediction

The most common match outcome is the binary indicator of win or loss, i.e., $y \in \{0, 1\}$, which is widely adopted in zero-sum games such as chess and DOTA2. In the traditional outcome prediction models, such as the Bradley-Terry model [3], each player’s overall proficiency is represented by a single scalar value γ . Then the probability of player a beating player b is modeled as:

$$\begin{aligned} P(a > b) &= \frac{\exp(\gamma_a)}{\exp(\gamma_a) + \exp(\gamma_b)} \\ &= \frac{1}{1 + \exp(-(\gamma_a - \gamma_b))} = \sigma(\gamma_a - \gamma_b) \end{aligned} \quad (2)$$

$$(3)$$

where γ_a is the skill rating of player a , $\sigma(x) = 1/(1 + \exp(-x))$ is the sigmoid function, reducing the model to logistic regression on pairs of individuals, and $P(a > b)$ is the predicted *win rate* for player a .

Another frequently encountered match outcome is adversarial scoring, which is usually adopted in team-oriented sports competitions, as noted in the work of [12]. In this work, we use the win rate as the prediction target for the outcome of a match since it has a wider range of applications [5, 6, 8]. However, our proposed method can be easily transferred to scenarios where match scores are available.

⁴More generally, we can seek to maximize the overall player retention, or equivalently minimize the overall player churn risk. However, that is beyond the scope of this paper.

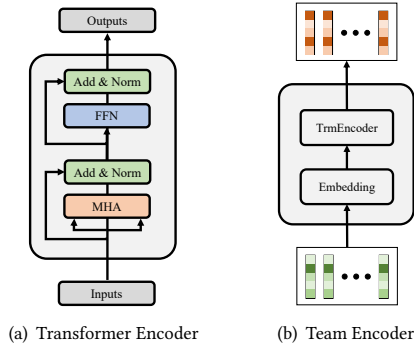


Figure 2: The Transformer Encoder and Team Encoder.

4.2 Balance Modelling

In general, we would assume that fair (balanced) matches lead to better game experience [8, 12, 13]. Following this intuition, we also consider the match balance as the main quality criterion of the matchmaking system. To this end, we utilize the outcome prediction introduced above to model the match balance between two competing teams, where the idea is that a match is balanced when the win rate is close to 50%.

For outcome prediction in the team-oriented game scenario, previous efforts usually learn an aggregated representation by naive pooling operation, such as summation or concatenation, over all players involved in the teams. e.g., the TrueSkill [13] system calculate a team rating by adding up skill ratings for all team members, while the models proposed in [8] compute team features as the sum of all player features in the team. In this work, instead, we propose a *Team Encoder* (TE) network to capture the intra-team interactions. It is inspired by the self-attentive mechanism in the Transformer [26] architecture, whose basic building layer consists of two sub-layers: the Multi-Head Attention (MHA) and the Feed-Forward Network (FFN).

The MHA utilizes a parallel attention mechanism, and the input consists of queries and keys of dimension d_k , and values of dimension d_v . We compute the dot products of the query with all keys, divide each by $\sqrt{d_k}$, and apply a softmax function to obtain the weights on the values. In practice, we compute the matrix of outputs as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V \quad (4)$$

$$\text{MHA}(Q, K, V) = \text{Concat}(\text{Head}_1, \dots, \text{Head}_h)W^O \quad (5)$$

$$\text{where Head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Where the matrices $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ and $W^O \in \mathbb{R}^{d_{\text{model}} \times d_k}$ are projection parameters for the MHA. As noted by [26], for each of these heads, we use $d_k = d_v = d_{\text{model}}/h$. The fully connected feed-forward network (FFN) consists of two linear transformations with a ReLU activation in between.

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (6)$$

where W_1, W_2, b_1 and b_2 are learnable parameters. In addition to the two sequential sub-layers, the basic layer of Transformer (we call it the *TrmEncoder* in this paper) also employs a residual connection around each of the two sub-layers, followed by layer normalization, as shown in Figure 2(a).

For a team of players $T = \{p_1, p_2, \dots, p_K\} \subset \mathcal{P}$ with their profile vectors $X = \{x_1, x_2, \dots, x_K\} \subset \mathcal{F}$, we compute the hidden representation by the Team Encoder (TE) as:

$$\text{TE}(T|X) = \text{TrmEncoder}(\text{Embedding}(X)) \quad (7)$$

where the *Embedding* operation is responsible for converting the input profile vector to a consistent hidden space $\mathbb{R}^{d_{\text{model}}}$, as shown in Figure 2(b).

Finally, given a pair of competing teams T_a and T_b , the (estimated) match balance is computed by a single-layer fully connected network with the concatenated outputs of team encoder.

$$H = \text{Concat}(\text{TE}(T_a), \text{TE}(T_b)) \quad (8)$$

$$\text{Balance}(T_a, T_b) = \sigma(w^T H + b) \quad (9)$$

where σ is the sigmoid activation function, and $w \in \mathbb{R}^{2d_{\text{model}}}$ and $b \in \mathbb{R}$ are learnable parameters of the model. We name the whole **Balance**($\cdot, \cdot$) for evaluating the match balance as **MBNet** (Match Balance Network).

5 GLOBALLY OPTIMIZED MATCHMAKING

To achieve a globally optimal matchmaking system, we consider the Markov Decision Process (MDP) formulation of the problem, in which the optimal solution can be viewed as a sequence of decisions. We regard the match balance model described above as the reward signal, aiming to generate near-optimal solutions by increasing the probability of finding desirable player match sequences.

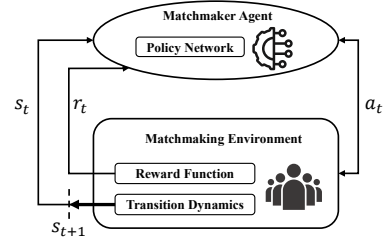


Figure 3: Demonstration of the interaction between the matchmaking environment and agent (matchmaker).

5.0.1 Reinforcement Learning Formulation. We formalize the global matchmaking problem as a sequential decision problem (Figure 3), and define the underlying MDP of the reinforcement learning formulation $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, P, r, p_o, \gamma \rangle$ as follows:

- **State space $\mathcal{S}$:** a state $s_t \in \mathcal{S}$ consists of two parts, one is the current player pool $\mathcal{P}_t$, and the other is the match context (currently selected players for both team A and team B) $A_t = \{pa_1, pa_2, \dots, pat\}$ and $B_t = \{pb_1, pb_2, \dots, pbt\}$, i.e., $s_t = (A_t, B_t, \mathcal{P}_t)$. The initial state $s_0 = (\emptyset, \emptyset, \mathcal{P})$, and the terminal state is activated when player pool $\mathcal{P}_t$ becomes empty

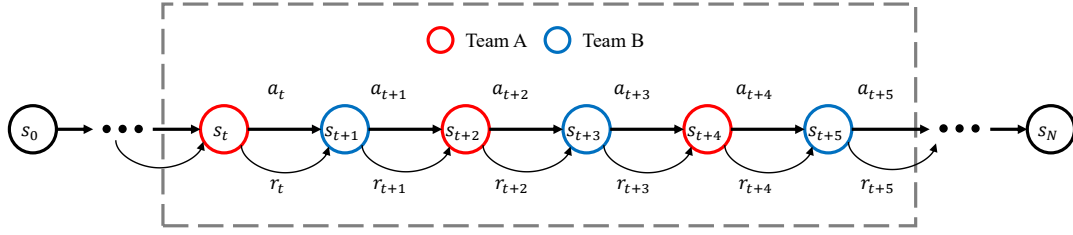


Figure 4: An Illustration of the MDP in the globally optimized matchmaking problem. The dashed box indicates the composition of a match. Without loss of generality, we take turns to select players for the competing teams and arrange the neighboring teams in the player sequence into one match.

$|\mathcal{P}_t| = 0$. Besides, we clear both A_t and B_t when finishing a matchup at every $2K$ steps.

- *Action space $\mathcal{A}$* : the set of candidate players in the matching pool $\mathcal{P}$, and the action $a_t \in \mathcal{P}_t$ corresponds to selecting a player from the matching pool that has not been selected currently.
- *Transition dynamics P* : the transition between states is deterministic here, and corresponds to adding a player $p \in \mathcal{P}_t$ that was selected as the last action to the current partial solution, i.e., $s_{t+1} = (A_t \cup \{p\}, B_t, \mathcal{P}_t \setminus \{p\})$ or $s_{t+1} = (A_t, B_t \cup \{p\}, \mathcal{P}_t \setminus \{p\})$, depending on which team is currently picking players.
- *Reward function r* : the (expected) reward function is described in the following section. Roughly speaking, we need a reward function that can help the agent learn to build matches that consider the balance requirements.
- *Initial probability p_0* : $\mathcal{A} \rightarrow [0, 1]$ is the probability distribution of initial states will depend on the matchmaking context.
- *Discount rate $\gamma \in [0, 1]$* : the discount factor is used to trade-off the importance of immediate and future rewards. We use $\gamma = 1$ in this work (i.e., the future reward is undiscounted).

In the sequential decision making setup, the agent interacts with the environment $\mathcal{E}$ over discrete time steps. In our case, selecting an action a in each step corresponds to adding a player to the current partial solution, which results in collecting a reward $r(s, a)$, as shown in Figure 4. We define the discounted return at time t as:

$$R_t = \sum_{i=t}^T \gamma^{i-t} r_i \quad (10)$$

where T is the time-step at which the matchmaking process terminates. The goal of the agent is to maximize the expected discounted return R_0 .

5.1 Player Constraint Graph

In practical scenarios, in order to ensure the player's gaming experience, there are usually some mandatory matching rules that need to be followed. e.g., the allowed skill gap between players, the network latency, etc. In order to take these factors into account, we introduce a player constraint graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, in which each node $v_i \in \mathcal{V}$ represents an individual player $p_i \in \mathcal{P}$ in the candidate player pool, and the edges $\mathcal{E}$ indicate that the player pairs

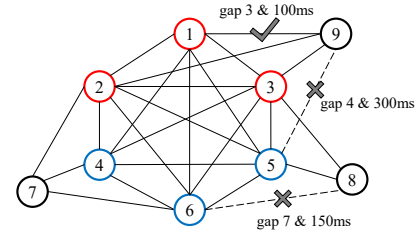


Figure 5: Illustration for the constraint graph. Nodes represent players, and edges represent that the players meet the matching rules.

meet the matching rules. An illustration is shown in Figure 5. For example, for the matching rules {skill gap ≤ 5 , network latency ≤ 200 ms}, players 1 and 9 can be matched together, players 5 and 9 should not be matched together because of the slow network connection, and players 6 and 8 either because of the excessive skill gap. Finally, a possible matchup result is the team $\{1, 2, 3\}$ versus the team $\{4, 5, 6\}$.

From a perspective of graph theory, the players involved in a match of the matchmaking result actually constitute a clique⁵. As such, at each decision step during the matching process, we restrict the available players to the neighbors of the already selected players. The matchmaking policy aims to select a clique ($2K$ -clique for a team size K) from graph $\mathcal{G}$ where the combination of the selected players achieves the maximal objective, and the characteristics of clique can ensure the constraint rules are satisfied.

5.2 Reward Design

The main goal of our work is to build an optimal matching sequence that is comparable for all competing teams, providing players with a fair and enjoyable game experience. To this end, we design a special reward signal in our MDP, i.e., the match balance reward $\text{Balance}(T_a, T_b)$ for the two competing teams T_a and T_b .

$$\text{Reward}(T_a, T_b) = -|\text{Balance}(T_a, T_b) - 0.5| \quad (11)$$

⁵A clique is a subset of vertices of an undirected graph such that every two distinct vertices in the clique are adjacent.

where we subtract a 0.5 (drawn probability of the match) term from the model output, and the negative sign in front of the formula is because we prefer small values for this term.

Here, the balance reward, which represents the balance degree between two competing teams, is meaningful only after completing the team composition for a match at every $2K$ steps. As such, the rewards are only given periodically during the matchmaking process. To be specific, given a state s_{t-1} , the matchmaking agent executes an action a_{t-1} and reaches the state $s_t = (A_t, B_t, \mathcal{P}_t)$ where $|A_t| = |B_t| = K$. Then, we define the corresponding reward $r(s_{t-1}, a_{t-1})$ as follows:

$$r(s_{t-1}, a_{t-1}) = \begin{cases} \text{Reward}(A_t, B_t) & \text{if } t = 2K, 4K, \dots \\ 0 & \text{otherwise} \end{cases}$$

where $\text{Reward}(\cdot, \cdot)$ is the reward function defined above. It is worth noting that our framework could be generalized to other quantifiable reward settings as well. Appendix B.3 provides a possible reward extension, which can explicitly consider the compatible abilities of teammates, and we leave it for future work.

5.3 Learning Algorithm

In this section, we introduce the policy network used to select an action given the current state. To learn a optimal matchmaking policy, we resort to the effective policy gradient (PG) algorithm [24] Proximal Policy Optimization (PPO) [22]. We choose to utilize PPO instead of value function methods like DQN [19, 29] since the policy representation in PG is more flexible than DQN and it usually has better learning convergence.

5.3.1 Policy Network. Generally, the policy of an agent in an MDP is a mapping from states to actions (or distributions over actions) $\pi : \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$. To learn a matchmaking policy π , we parameterize an approximate policy function $\pi(a|s; \theta)$ using the deep neural network, in which θ are the parameters of the policy network. As the core element of the PG algorithm, the design of policy networks is basically problem-specific. For our matchmaking problem, we resort to a specially designed attention network, which extends the Pointer Networks [1, 27] to the graph structure. In order to take into account the constraints between players described in section 5.1, at each decision step, we apply a *masked attention* on the player constraint graph to output the action probabilities corresponding to currently available players. The overall structure of the policy network $\pi(a|s; \theta)$ is shown in Figure 6.

Specifically, given a state $s_t = (A_t, B_t, \mathcal{P}_t)$ at each step, firstly, we apply the *Team Encoder* described in section 4.2 to the partial solution (already selected players for a match) A_t and B_t , resulting in a match context consisting of $\mathcal{H}_a = \text{TE}(A_t)$ and $\mathcal{H}_b = \text{TE}(B_t)$. We also learn a hidden representation $\mathcal{H}_{pool} = \{h_i \in \mathbb{R}^d | p_i \in \mathcal{P}_t\}$ for players in the current matching pool $\mathcal{P}_t$ by a *Pool Encoder*, which is similar to the *Team Encoder* except that using all candidate players' features as inputs. Then, we compute an aggregated representation for $\mathcal{H}_a$, $\mathcal{H}_b$ and $\mathcal{H}_{pool}$ by mean-pooling, i.e., $h_a = \frac{1}{|A_t|} \sum_{i \in A_t} h_i$, $h_b = \frac{1}{|B_t|} \sum_{i \in B_t} h_i$, $h_{pool} = \frac{1}{|\mathcal{P}_t|} \sum_{i \in \mathcal{P}_t} h_i$. Finally, h_a , h_b and h_{pool} are concatenated to obtain an context vector c_t , which is used to

get a softmax distribution over the players of the matching pool by attending to the encoded information $\mathcal{H}_{pool}$.

$$c_t = \text{Concat}(h_a, h_b, h_{pool}) \quad (12)$$

$$z_t^i = v^T \tanh(W_1 c_t + W_2 h_i), \quad p_i \in \mathcal{P}_t \quad (13)$$

$$\pi_\theta(a_t = i | s_t) = \text{softmax}(z_t^i), \quad p_i \in \mathcal{P}_t \quad (14)$$

where Concat is the concatenation operation, and v , W_1 and W_2 are learnable parameters of the pointer attention network. Note that we mixed some symbols when the context is clear.

5.3.2 Policy Learning. Policy gradient methods work by computing an estimator of the policy gradient and plugging it into a stochastic gradient ascent algorithm. For an agent behaving according to a stochastic policy π_θ , the most commonly used gradient estimator $\hat{g}$ has the form:

$$\hat{g} = \hat{\mathbb{E}}_t [\hat{A}_t \nabla_\theta \log \pi_\theta(a_t | s_t)] \quad (15)$$

where $\hat{A}_t(s_t, a_t)$ is an estimator of the advantage function at timestep t [22, 29], and is utilized to reduce the variance of policy gradient algorithms. Here, the expectation $\hat{\mathbb{E}}_t[\cdot]$ indicates the empirical average over a finite batch of samples.

To demonstrate the effectiveness of the proposed matchmaking framework, we first design a simulated experimental environment. After verifying the feasibility of the method, it is deployed into several industrial matchmaking services of popular online games released by *Netease Games*⁶: *Fever Basketball*⁷, a online basketball mobile game, and *Justice Online*⁸, which is a popular MMORPG running on the PC. The implementation details are provided in Appendix A.

6 SIMULATION EXPERIMENTS

To verify the ideas introduced above, we conduct several simulation experiments on industrial datasets to evaluate our framework, with several benchmark methods for experimental comparison.

Table 1: Summary Statistics of the Datasets

Dataset	Matches	Players	Features	Team Mode
Fever Basketball	466,529	30,040	22	3-vs-3
Justice Online	400,985	185,232	36	15-vs-15

6.1 Experimental Settings

We collect two large-scale industrial match datasets, which contains half a year of match records in the online games. The statistics of the dataset are briefly shown in Table 1. Here, the simulation experiments assume a matching pool of 120 candidate players for both of the *Fever Basketball* and *Justice Online* datasets, i.e., building 20 and 4 matches for these two datasets, respectively.

⁶<https://www.neteasegames.com/>

⁷<https://chao.163.com/>

⁸<https://n.163.com/>

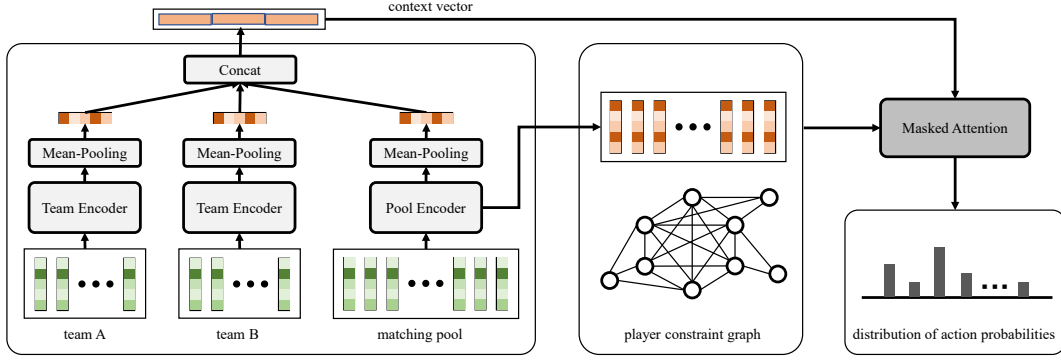


Figure 6: An illustration of the Policy Network $\pi_{\theta}(a|s)$.

6.1.1 Players Features. Since feature engineering is critical to the successful application of machine learning models, we calculate several features that imply the various abilities of players involved in the competition. These features are usually designed by the experts in the domain and are greatly case-specific. An example of the feature design that we implemented for the Fever Basketball dataset is given in the Appendix B.1.

6.1.2 Balance Modelling Baselines. In the simulated matchmaking environments, we use the estimated outcome prediction model as a balance reward for the matches. Therefore, the accuracy of the outcome prediction model is crucial to verify the effectiveness of the matchmaking methods. We compared the performance of following methods on the collected game datasets.

- **LR.** The logistic regression model.
- **BalanceNet.** A match outcome prediction model proposed in [8], which is basically a multi-layer perceptron (MLP) model with players' feature vectors as the input.
- **MBNet.** Our proposed balance model described in section 4.2, which has a subtle difference from our previous work in *OptMatch* [12], using a slightly different network structure and label setting.

Based on the collected match data and handcrafted features, we train these balance models through supervised learning, with the binary indicator of win or loss $y \in \{0, 1\}$ as the label. We use the binary cross entropy between the target label and the model output as the loss function.

$$L = \frac{1}{n} \sum_i^n -[y_i \log(p_i) + (1 - y_i) \log(1 - p_i)] \quad (16)$$

where n is the number of samples, y_i is the win or loss label for match i , and p_i is the predicted match balance formulated by equation 9. After training, a simulated matchmaking environment was constructed, where the output value of the balance model was used as an estimate of the actual reward signal according to equation 11.

6.1.3 Global Matchmaking Baselines. To compare the performance of global matchmaking for a pool of players, we include the following methods in the simulation experiments, along with a random policy for comparison purposes.

- **Random.** A random matchmaking strategy.
- **Heuristic.** We rank all players in the player pool according to the Elo ratings recorded in the game, and then arrange players with similar ratings into competition teams.
- **OptMatch.** A state-of-the-art two-stage matchmaking framework introduced in our previous work [12], which combines heuristic rules and an outcome prediction model. The rules is utilized to split queuing players into multiple matchup

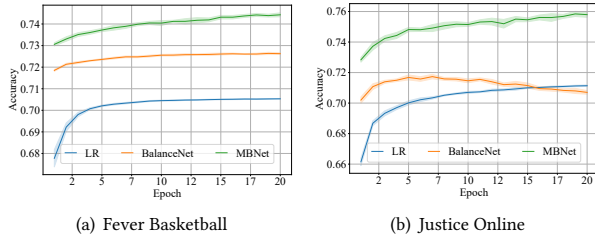


Figure 7: Learning curves of the match balance prediction model for two datasets. Lines show the mean values, while shades indicate the related standard deviations.

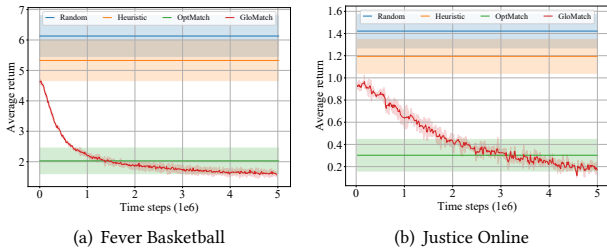


Figure 8: Performance curves of the matchmaking methods in the two simulated environments. The return refers to the *absolute* value of the cumulative reward (smaller value means better cumulative match balance). Lines show the mean values, while shades indicate the related standard deviations.

proposals, and the outcome prediction model is utilized to search the best player assignment for two competing teams.

- **GloMatch.** Our proposed framework in this paper.

6.2 Results and Analysis

The learning curves of the balance models on the two datasets are shown in Figure 7. From the reported results, we can find that our proposed outcome prediction model outperforms the baseline methods significantly, verifying the effectiveness of the proposed MBNet model. Using the learned balance model as an (estimated) reward function, we report the matchmaking performance on the average reward per episode on the simulation settings, which is summarized and shown in Figure 8. The return in the plot is the absolute value of the cumulative reward defined by equation 10. Note that the reward value defined by equation 11 is a negative number, and smaller absolute value means better cumulative match balance. The experimental results demonstrate that GloMatch not only effectively converges to the (local) optimum, but also has better matching quality in the simulated environments compared to other baselines. By these simulation experiments, we could better observe the model’s ability to perform the combinatorial search.

7 FIELD EXPERIMENTS

We deployed the proposed matchmaking system into the industrial matchmaking services in *Fever Basketball* and *Justice Online*. For the performance comparison with the original system, which is designed with a basic skill rating system and sophisticated experience rules, we conducted online A/B testing. To this end, we randomly distribute matchmaking requests to two channels: the original system and the proposed GloMatch system.

7.1 Evaluation Metrics

Following the assumptions made in the existing literature that fair (balanced) matches bring players a better gaming experience [8, 12, 13], in this work, we use the Score Difference (SD), which indicates the difference between the final match scores of two competing teams, to evaluate the online performance of matchmaking methods. The idea is that a match is balanced when the score difference is close to zero.

$$SD = \frac{1}{n} \sum_{i=1}^n |s_a^i - s_b^i| \quad (17)$$

where n is the number of matches, s_a^i is the match score for team T_a in match i , and $|\cdot|$ computes the absolute value.

7.2 Offline Pre-training

The training of the RL algorithm requires the agent to trade-off exploration and exploitation, which means that many early actions of the agent may be randomly generated during the learning process. It is risky to deploy it online directly, since these random matching results will seriously harm the player’s gaming experience. To alleviate this problem, we designed an offline pre-training module. It is basically the same as the process we introduced in the simulation experiment part, i.e., we collect a large enough online match log data in advance, and then use these data to train an estimator of

the match balance, whose output is used as a reward signal to train the matchmaking agent.

7.3 Online Fine-tuning

Table 2: Overall performance comparison of online match-making systems on the two games (smaller value refers to better match quality).

Method	<i>Fever Basketball</i>	<i>Justice Online</i>
Original	7.20 ± 5.59	1314.84 ± 872.41
OptMatch	6.82 ± 5.15	1202.54 ± 820.88
GloMatch	6.67 ± 5.17	1145.18 ± 815.59
Improve(%)	7.36	12.90

After verifying the feasibility and finishing the offline pre-training, we launched the system into the game, and further fine-tune the matchmaking policy according to the actual online matching environment. Appendix B.2 gives a detailed online reward design, which is adjusted to be consistent with the reward in the simulation experiments. To evaluate the online performance, we compare the (absolute) score difference defined in equation 17 for the matches. Table 2 shows the statistics for more than three months since the system was deployed (more than 100,000 matches involved), from which we can find that the matches assigned by GloMatch have generally smaller score difference, which means fairer games and better matching quality.

7.4 System Performance

The single inference time of our policy network is about 2~3 milliseconds with a GPU device and 20~30 milliseconds with a CPU device. In terms of the system response speed, according to the online service logs, a global matchmaking request usually involves less than 200 players. Overall, the GloMatch system takes no more than 600ms in average to respond to one matchmaking request with a GPU hardware resource. When more players are involved in a match (e.g., 15 in the game *Justice Online*), our method is much faster than the two-stage methods, such as OptMatch which in this case can only search a very small combination space because of the limited time requirement.

8 CONCLUSION AND FUTURE WORK

In this paper, we target at a practical but less explored matching problem named globally optimized matchmaking, where the objective is to decide an optimal matching sequence for the queuing players. Unlike the traditional skill rating system, which focuses on evaluating the player’s true skill level through comparative data, our framework specifically addresses the optimization problem of gross player utilities (i.e., satisfaction) for a pool of players. To tackle this challenging problem in the context of matchmaking for online games, we introduce an end-to-end globally optimized matchmaking framework. Specifically, we formalize the matchmaking process as a Markov Decision Process, which is specially designed for our globally optimized matchmaking problem, and we further solve

the problem by an effective policy-based deep reinforcement learning algorithm. Quantitative experiments on simulation and online game environments demonstrate the effectiveness of the presented framework.

In the future, in addition to the win/loss balance of the match, more optimization objectives can be considered, such as minimizing the average waiting time of players. One possible approach to deal with this type of goal is to incorporate the player's waiting time into the reward function, which can consider players who are always in the player pool and have been waiting for a long time. Finally, although only two online games were evaluated in this work, it would be interesting to adapt the proposed framework to similar problems in other types of games.

REFERENCES

- [1] Irwan Bello, Hieu Pham, Quoc V Le, Mohammad Norouzi, and Samy Bengio. 2016. Neural combinatorial optimization with reinforcement learning. *arXiv preprint arXiv:1611.09940* (2016).
- [2] Alexander J Bisberg and Rogelio E Cardona-Rivera. 2019. SCOPE: Selective Cross-Validation over Parameters for Elo. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, Vol. 15. 116–122.
- [3] Ralph Allan Bradley and Milton E Terry. 1952. Rank analysis of incomplete block designs: I. The method of paired comparisons. *Biometrika* 39, 3/4 (1952), 324–345.
- [4] Shuo Chen and Thorsten Joachims. 2016. Modeling intransitivity in matchup and comparison data. In *Proceedings of the ninth acm international conference on web search and data mining*. 227–236.
- [5] Shuo Chen and Thorsten Joachims. 2016. Predicting matchups and preferences in context. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 775–784.
- [6] Zhengxing Chen, Su Xue, John Kolen, Navid Aghdaie, Kazi A Zaman, Yizhou Sun, and Magy Seif El-Nasr. 2017. Eomm: An engagement optimized matchmaking framework. In *Proceedings of the 26th International Conference on World Wide Web*. 1143–1150.
- [7] Pierre Dangauthier, Ralf Herbrich, Tom Minka, and Thore Graepel. 2008. Trueskill through time: Revisiting the history of chess. In *Advances in neural information processing systems*. 337–344.
- [8] Olivier Delalleau, Emile Contal, Eric Thibodeau-Laufer, Raul Chandias Ferrari, Yoshua Bengio, and Frank Zhang. 2012. Beyond skill rating: Advanced matchmaking in ghost recon online. *IEEE Transactions on Computational Intelligence and AI in Games* 4, 3 (2012), 167–177.
- [9] Arpad E Elo. 1978. *The rating of chessplayers, past and present*. Arco Pub.
- [10] Mark E Glickman. 1995. A comprehensive guide to chess ratings. *American Chess Journal* 3, 1 (1995), 59–102.
- [11] Mark E Glickman. 1999. Parameter estimation in large dynamic paired comparison experiments. *Journal of the Royal Statistical Society: Series C (Applied Statistics)* 48, 3 (1999), 377–394.
- [12] Linxia Gong, Xiaochuan Feng, Dezhi Ye, Hao Li, Runze Wu, Jianrong Tao, Changjie Fan, and Peng Cui. 2020. OptMatch: Optimized Matchmaking via Modeling the High-Order Interactions on the Arena. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2300–2310.
- [13] Ralf Herbrich, Tom Minka, and Thore Graepel. 2007. TrueSkill™: a Bayesian skill rating system. In *Advances in neural information processing systems*. 569–576.
- [14] Tzu-Kuo Huang, Chih-Jen Lin, and Ruby Weng. 2004. A generalized Bradley-Terry model: From group competition to individual skill. *Advances in neural information processing systems* 17 (2004), 601–608.
- [15] Tzu-Kuo Huang, Chih-Jen Lin, and Ruby C Weng. 2008. Ranking individuals by group comparisons. *Journal of Machine Learning Research* 9, Oct (2008), 2187–2216.
- [16] Diederik P Kingma and Jimmy Ba. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* (2014).
- [17] Yao Li, Minhao Cheng, Kevin Fujii, Fushing Hsieh, and Cho-Jui Hsieh. 2018. Learning from group comparisons: exploiting higher order interactions. In *Advances in Neural Information Processing Systems*. 4981–4990.
- [18] Tom Minka, Ryan Cleven, and Yordan Zaykov. 2018. Trueskill 2: An improved bayesian skill rating system. *Tech. Rep.* (2018).
- [19] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. 2015. Human-level control through deep reinforcement learning. *nature* 518, 7540 (2015), 529–533.
- [20] Mohammadreza Nazari, Afshin Oroojlooy, Lawrence Snyder, and Martin Takác. 2018. Reinforcement learning for solving the vehicle routing problem. In *Advances in Neural Information Processing Systems*. 9839–9849.
- [21] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. Pytorch: An imperative style, high-performance deep learning library. In *Advances in neural information processing systems*. 8026–8037.
- [22] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* (2017).
- [23] Aleksandr Semenov, Peter Romov, Sergey Korolev, Daniil Yashkov, and Kirill Neklyudov. 2016. Performance of machine learning algorithms in predicting game outcome from drafts in Dota 2. In *International Conference on Analysis of Images, Social Networks and Texts*. Springer, 26–37.
- [24] Richard S Sutton, David A McAllester, Satinder P Singh, and Yishay Mansour. 2000. Policy gradient methods for reinforcement learning with function approximation. In *Advances in neural information processing systems*. 1057–1063.
- [25] Louis L Thurstone. 1927. A law of comparative judgment. *Psychological review* 34, 4 (1927), 273.
- [26] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in neural information processing systems*. 5998–6008.
- [27] Oriol Vinyals, Meire Fortunato, and Navdeep Jaitly. 2015. Pointer networks. In *Advances in neural information processing systems*. 2692–2700.
- [28] Kai Wang, Hao Li, Linxia Gong, Jianrong Tao, Runze Wu, Changjie Fan, Liang Chen, and Peng Cui. 2020. Match Tracing: A Unified Framework for Real-time Win Prediction and Quantifiable Performance Evaluation. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*. 2781–2788.
- [29] Ziyu Wang, Tom Schaul, Matteo Hessel, Hado Hasselt, Marc Lanctot, and Nando Freitas. 2016. Dueling network architectures for deep reinforcement learning. In *International conference on machine learning*. PMLR, 1995–2003.

Table 3: Handcrafted features for players in the *Fever Basketball* dataset.

Feature No.	Feature Description
1	role level of the player account
2	average assistance
3	average times of being blocked
4	average passes
5	average times of being stolen
6	average times of ball touching
7	average blocks
8	total game times
9	total win game times
10	total lose game times
11	average times of mentor guidance
12	the mvp rate
13	the overtime rate
14	average rebounds
15	average scores by player
16	statistics of skill uses
17	average steal times
18	average 3-point fieldgoals
19	average 3-point shoots
20	average 2-point fieldgoals
21	average 2-point shoots
22	average wideopen scores

A IMPLEMENTATION DETAILS

We implement the match outcome prediction model and policy network by PyTorch [21] with the Adam optimizer [16].

- We employ early stopping and dropout techniques to prevent over-fitting when possible.
- The embedding size is fixed to 128 for all models. The model dimension d_{model} , which refers to the input and output dimension of the *TrmEncoder* (equation 7), is also fixed to 128. The hidden sizes in FFN (equation 6) and policy network (equation 13) are set to 512 and 128, respectively.
- In this work, the team encoder is composed of a stack of $L = 1$ identical self-attention layers in addition to the first embedding layer, and we employ $h = 2$ parallel attention heads.
- We apply grid search for tuning the hyper-parameters of the outcome prediction model: the learning rate is tuned amongst $\{0.0001, 0.0005, 0.001, 0.005, 0.01\}$, the coefficient of $L2$ regularization is searched in $\{10^{-5}, 10^{-4}, \dots, 1\}$, and the dropout ratio in $\{0.0, 0.1, \dots, 0.5\}$. The batch size is fixed to 512 for all models. The set of possible hyper-parameter values was determined on early validation tests using subsets of the datasets which are then discarded from our analyses.
- For the OptMatch model in the simulation experiment, we give it a search time of 10s, which is basically much longer than the allowable time of the online system, and then use the optimal solution as the matching result of a single match.

B RESEARCH METHODS

B.1 Feature Design

For the game datasets used in the simulation experiments, we designed several features related to the game characteristics. As an example, Table 3 lists the handcrafted features we built for players in *Fever Basketball* dataset alongside their descriptions.

B.2 Online Reward Design

In the simulated environment introduced in the experimental section, we use the estimated match balance to provide a reward signal for the matchmaking agent. However, in the real online game environment, there is no such information that directly corresponds to the balance of the match. The most relevant information we can perceive is the score difference between the competing teams.

In this case, for two teams T_a and T_b with scores s_a and s_b respectively, a straightforward approach is to consider the value of the absolute score difference $y = \pm |s_a - s_b| \in \mathbb{R}$ as the reward, where the sign is determined by which team it refers to (+1 for the winning team and -1 for the losing one). However, this may lead to two limitations: first, the magnitude of the score is diversified and may be tens to hundreds, resulting in the reward value not suitable for feeding directly to the neural network models; second, it cannot well reflect the competition intensity of the match, e.g., the 90 to 100 match is generally more intense and exciting than the 0 to 10 match, although both of their score difference is 10. In order to get a consistent reward signal for online fine-tuning, we made the following transformations to the match scores.

$$o = \pm \left| \frac{s_a - s_b}{s_a + s_b} \right| \quad (18)$$

$$\text{Balance}(T_a, T_b) = \frac{1}{2}(o + 1) \quad (19)$$

where s_a is the match score for team T_a , $|\cdot|$ represents the absolute value, and $o \in [-1, 1]$ is the relative outcome based on the score difference.

B.3 Compatibility Reward Extension

Although match balance is an obvious criterion for measuring the matchmaking quality. In some cases, it may be limited to consider only this criteria. For example, in an FPS game, having two teams of *campers* (players who tend to stay still and wait to ambush enemies) will most likely lead to a boring match where no action ever happens, even if the match is perfectly balanced.

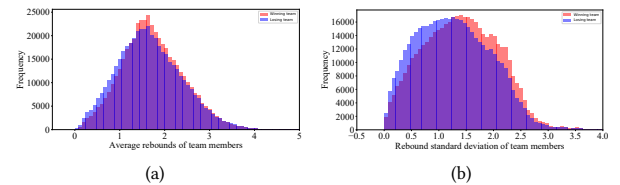


Figure 9: Distribution of the number of rebounds between the winning and losing teams. (a) Mean value of rebounds. (b) Standard deviation of rebounds.

Intuitively, it is better for members of a team to have *compatible* abilities rather than *similar* abilities, because multiple member players with different advantages can make the team’s capabilities more comprehensive and more likely to gain team synergy effect in the competition, providing players with an more enjoyable game experience. However, compared with the balance factor, the compatibility factor is difficult to clearly define and directly measure.

To incorporate such prior knowledge into the matchmaking system, for example, we could define the player compatibility degree as the diversity of player abilities within the teammates, which is measured by the variance of the attribute values of teammates. Figure 9 is a distribution of the number of rebounds between the winning and losing teams in *Fever Basketball* (an online basketball game). Interestingly, although the distribution of average rebounds is very similar, the winning team and the losing team show a certain deviation in the standard deviation of the attribute value.

To be specific, given a team T of player profile vectors $X = \{x_1, x_2, \dots, x_K\}$, we can compute the compatibility degree of the teammates as:

$$\text{Compat}(T) = \frac{1}{|F|} \sum_{f \in F} \text{Var}(f|T, X) \quad (20)$$

where F is the set of feature dimensions, and $\text{Var}(f|T, X)$ denotes the variance of a feature f for all teammates. Although the reward function can be defined like this, the diversity of abilities

among players is only designed based on intuitive human experience, which in most cases is impossible to get a quantifiable indicator. It may require some *empirical research* methods to prove its effectiveness to improve the subjective satisfaction of players, which is beyond the focus of this paper.

C ONLINE SYSTEM DATAFLOW

Figure 10 shows the overall dataflow of our matchmaking system, where the arrows indicate the data flow and the interaction between the system components.

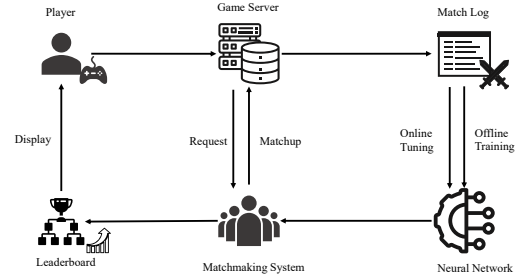


Figure 10: Overview of the Matchmaking System.